CSE574-D: Introduction to Machine Learning, Fall 2025

Instructor: Dr. Alina Vereshchaka

Assignment 3

Defining and Solving Reinforcement Learning Task

Checkpoint: Nov 20, Thu, 11:59pm

Due Date: Dec 4, Thu, 11:59pm

Welcome to the third assignment for this course. The goal of this assignment is to acquire experience in defining and solving a reinforcement learning (RL) environment, following Gymnasium standards.

The assignment consists of three parts. The first part focuses on defining an environment that is based on a Markov decision process (MDP). In the second part, we apply a tabular method SARSA to solve an environment. In the third part, we extend the solution and apply the n-step Double Q-learning method to solve an environment.

Part I: Define an RL Environment [30 pts]

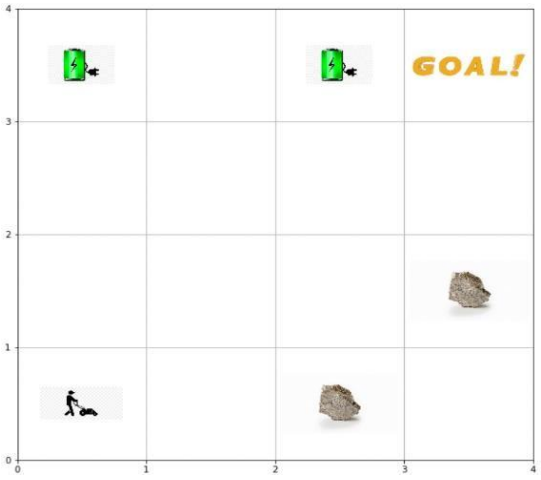
In this part, we will define a grid-world reinforcement learning environment as an MDP. While building an RL environment, you need to define possible states, actions, rewards and other parameters.

STEPS:

1. Choose a scenario for your grid world. You are welcome to use the RL env visualization demo as a reference to visualize it.

An example of idea for RL environment:

- **Theme**: Lawnmower Grid World with batteries as positive rewards and rocks as negative rewards.
- **States:** {S1 = (0,0), S2 = (0,1), S3 = (0,2), S4 = (0,3), S5 = (1,0), S6 = (1,1), S7 = (1,2), S8 = (1,3), S9 = (2,0), S10 = (2,1), S11 = (2,2), S12 = (2,3), S13 = (3,0), S14 = (3,1), S15 = (3,2), S16 = (3,3)}
- **Actions**: {Up, Down, Right, Left}
- **Rewards**: {-5, -6, +5, +6}
- **Objective**: Reach the goal state with maximum reward



2. Define an RL environment following the scenario that you chose.
   Environment requirements:
   - Min number of states: 20
   - Min number of actions: 4
   - Min number of rewards: 4

   Environment definition should follow the Gymnasium structure, which includes the basic methods. You can use RL Environment demo as a base code.

```
def __init__:
     # Initializes the class
     # Define action and observation space

def step:
     # Executes one timestep within the environment
     # Input to the function is an action

def reset:
     # Resets the state of the environment to an initial state

def render:
     # Visualizes the environment
     # Any form like vector representation or visualizing
     usingmatplotlib is sufficient
```

3. Run a random agent for at least 10 timesteps to show that the environment logic is defined correctly. Print the current state, chosen action, reward and return your grid world visualization for each step.
4. Describe the environment that you defined. Provide a set of states, actions, rewards, main objective, etc.
5. Provide visualization of your environment.
6. Safety in AI: Write a brief review (~ 5 sentences) explaining how you ensure the safety of your environment. E.g. how do you ensure that the agent chooses only actions that are allowed, that agent is navigating within defined state-space, etc
7. [For teams of 2] Include a contribution summary for each team member in the format:

Step	Teammate	Contribution %

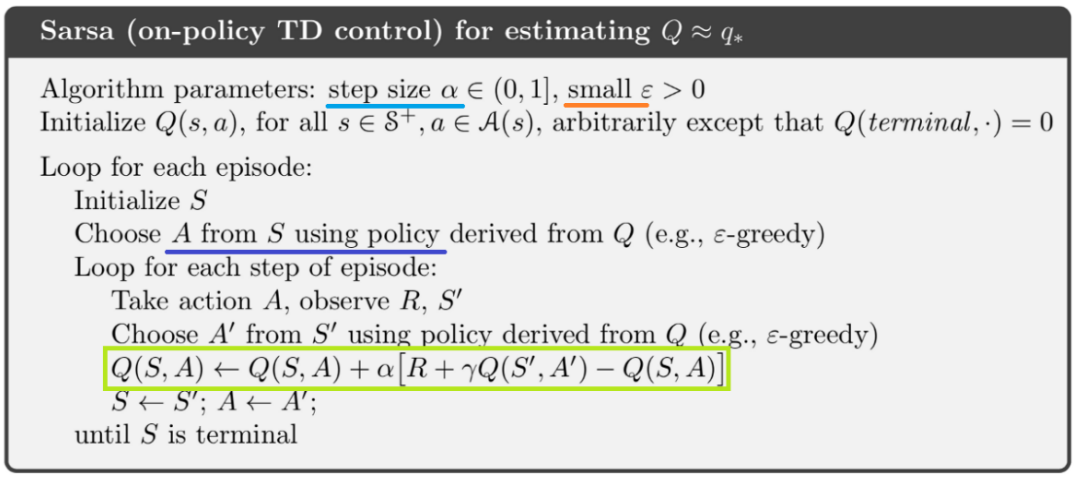
Part II: Implement SARSA [30 pts]

In this part, you will implement the SARSA (State-Action-Reward-State-Action) algorithm and apply it to solve the environment defined in Part I.

SARSA is an on-policy reinforcement learning algorithm where the agent updates its Q-values based on the current state, action, reward, and next state-action pair. It balances exploration and exploitation to optimize learning.

STEPS:

1. Implement the SARSA algorithm to solve the environment defined in Part I.



Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

2. Provide the evaluation results:
    a. Print the initial Q-table and the trained Q-table
    b. Plot the total reward per episode graph (x-axis: episode, y-axis: total reward per episode).
    c. Plot the epsilon decay graph (x-axis: episode, y-axis: epsilon value)
    a. Greedy policy evaluation: run your environment for at least 10 episodes, where the agent chooses only greedy actions from the learned policy. Include a plot of the total reward per episode.
3. Hyperparameter tuning. Select at least two hyperparameters for better SARSA performance. You can explore hyperparameter tuning libraries, e.g. Optuna or perform manual tuning. Parameters to tune (select 2):
    - Discount factor (γ)
    - Epsilon decay rate
    - Epsilon min/max values
    - Number of episodes
    - Max timesteps

    Try at least 3 different values for each of the parameters that you choose.

4. Provide a short description on how these hyperparameters influence performance. Suggest the most efficient hyperparameter values for your problem setup. Combine all the parameters that return the best results, e.g. faster training, better performance and use it as the best model.
5. Provide the evaluation results (refer to Step 2) for the setup the returns the best results. Provide the description, plots and evaluation results for the setup the returns the best results (Step 4). Provide your detailed analysis.
6. [For teams of 2] Include a contribution summary for each team member in the format:

Step	Teammate	Contribution %

Part III: Implement N-step Double Q-learning [40 pts]

Apply the n-step Double Q-learning algorithm to solve the environment defined in Part I. This will involve both theoretical understanding and practical implementation.

STEPS:

1. **[Theoretical Task]** Derive the n-step Double Q-Learning update rule:
    - Mathematically derive the update rule for the n-step Double Q-learning algorithm. Include the details in your report.
    - Consider n values ranging from 1 to 5 and provide a generalizable update rule.

      Refer to Sutton & Barto's Reinforcement Learning book, Chapters 6.1 and 7 for details.
2. **[Practical Implementation]** Implement the n-step Double Q-learning algorithm to solve the environment defined in Part I.
    - Modify your SARSA implementation to incorporate the n-step Double Q-learning update rule.
3. Experiment with hyperparameters to further optimize n-step Double Q-learning. Parameters to try include discount factor (γ), epsilon decay rate, epsilon min/max values, number of episodes, and max timesteps. Return the best model setup. Provide a short description on how hyperparameters influence performance. Suggest the most efficient hyperparameter values for your problem setup
4. Provide the results with different values of n (1, 2, 3, 4, 5). Determine the optimal n-step return for your environment:
    - Print the initial and trained Q-tables for each value of n.
    - Plot the total reward per episode for each value of n.
    - Plot the epsilon decay graph for each value of n.

- Run the environment for at least 10 episodes using only greedy actions for each value of n. Plot the total reward per episode.

Provide the evaluation results and your explanation. Suggest the most efficient n value and hyperparameter setup for your problem.

5. Compare the performance of SARSA and n-step Double Q-learning algorithms on the same environment (e.g. show one graph with two reward dynamics) and give your interpretation of the results.
6. [For teams of 2] Include a contribution summary for each team member in the format:

Step	Teammate	Contribution %

Bonus task [max 8 points]

Grid-World Environment Visualization [3 points]

Add custom-defined images into your grid world env to represent:

- Agent: at least two images dependent on what the agent is doing
- Background: a setup that represents your scenario (different from the default one)
- Images representing each object in your scenario
- Run your environment for at least 10 episodes with show the dynamic of your environment

You can refer to Matplotlib for RL Env Visualizing demo to help you get started.

SUBMIT:

1. Provide the details of your scenario and visualizations as part of your report
2. Submit Jupyter notebook with images named as a3_bonus_visualization_TEAMMATE1_ TEAMMATE2.ipynb

Defining & Solving Warehouse Robot Environment [5 points]

Define a new RL environment following the scenario below and solve it using SARSA. You are welcome to base your implementation based on Part I & Part II.

STEPS:

1. Define Warehouse Robot environment.

   Scenario: A robot operates in a warehouse grid where its task is to pick up items from specified locations and deliver them to designated drop-off points. The warehouse has shelves that act as obstacles, and the robot must navigate around

them efficiently.

Environment setup:

- **Grid size:** 6x6 grid
- **Obstacles:** Shelves (static obstacles) that block the robot's path
- **Goal:** The robot must pick up an item from an assigned location and deliver it to another specified location
- **Actions:** Up, Down, Left, Right, Pick-up, Drop-off
- **Rewards:**
  - +100pts for successful delivery
  - +25 points ONLY for picking up the object the first time
  - -1pt for each step taken to encourage efficiency
  - -20pts for hitting an obstacle
- **Terminal state:** The robot successfully delivers the item

Tips:

- **Observation space:** The agent needs additional information beyond its location in the environment to solve it. Provided that there's only a single object in the environment and it's always in the same location when the environment is reset, the following observation space should suffice:

  {([0, 0], False), ([0, 1], False), ..., ([0, 4], False), ([0, 5], False), ([0, 0], True), ([0, 1], True), ..., ([0, 4], True), ([0, 5], True), ..., ([5, 0], False), ([5, 1], False), ..., ([5, 4], False), ([5, 5], False), ([5, 0], True), ([5, 1], True), ..., ([5, 4], True), ([5, 5], True)}

  Here, the first element of the tuple is the agent's coordinates. The second element is a boolean that indicates whether it is carrying the object. Please note that this observation space is the minimum you'll need to be able to solve the environment. The number of unique observations will be 2 * number of grid blocks.

- **Q-Table structure:** You can use a dictionary with the keys being the grid positions.

Requirements:

- The robot (RL Agent) should be able to navigate from any start position to any target position.
- Implement collision detection to prevent the robot from moving through shelves.

2. Run a random agent for at least 10 timesteps to show that the environment logic is defined correctly. Print the current state, chosen action, reward and return your grid world visualization for each step.
3. Solve this environment using SARSA. Provide the evaluation results:
   a. Print the initial Q-table and the trained Q-table

b. Plot the total reward per episode graph (x-axis: episode, y-axis: total reward per episode).
c. Plot the epsilon decay graph (x-axis: episode, y-axis: epsilon value)
d. Greedy policy evaluation: run your environment for at least 10 episodes, where the agent chooses only greedy actions from the learned policy. Include a plot of the total reward per episode.

SUBMIT:

1. Submit Jupyter notebook with saved results as a3_bonus_warehouse_TEAMMATE1_ TEAMMATE2.ipynb

Assignment Steps

1. Register your team (Nov 12)

Register your team at UBLearns > Groups. You may work individually or in a team of up to 2 people. The evaluation will be the same for a team of any size.

Your teammates shouldbe different for A1, A2 & A3. If you joined the wrong group, make a private post on piazza.

2. Submit checkpoint (Nov 20)

- Complete Part I and Part II of the assignment.
- Include all the references at the end of each part.
- Your Jupyter notebook should be saved with the outputs.
- If you are working in a team, we expect equal contribution for the assignment. Each team member is expected to make a code-related contribution. Provide a contribution summary by each team member in the form of a table below. If the contribution is highly skewed, then the scores of the team members may be scaled w.r.t the contribution.

Step	Teammate	Contribution (%)

- Combine all files in a single zip folder that will be submitted on UBlearns, named as assignment_3_checkpoint_YOUR_UBIT.zip
- Submit to UBLearns > Assignments
- Suggested file structure:

  assignment_3_checkpoint_TEAMMATE1_ TEAMMATE2.zip

- a2_part_1_ TEAMMATE1_ TEAMMATE2.ipynb
- a2_part_2_ TEAMMATE1_ TEAMMATE2.ipynb

3. Submit final results (Dec 4)

- Complete all parts of the assignment (I-III, plus Bonus if attempted).
- Add all your assignment files in a zip folder including ipynb files for Part I, Part II, Part III & Bonus part (optional).
- Jupyter Notebooks must contain saved outputs and plots visible.
- Include all references at the end of each part.
- You may make unlimited submissions, only the latest will be evaluated.
- Name zip folder with all the files as assignment_3_final_ TEAMMATE1_ TEAMMATE2.zip
  e.g. assignment_3_final_avereshc_manasira.zip
- Submit to UBLearns > Assignments
- Your Jupyter notebook should be saved with the outputs.
- Include all the references at the end of each part that have been used to complete that part.
- You can make unlimited number of submissions and only the latest will be evaluated
- If you are working in a team, we expect equal contribution for the assignment. Each team member is expected to make a code-related contribution. Provide a contribution summary by each team member in the form of a table below. If the contribution is highly skewed, then the scores of the team members may be scaled w.r.t the contribution.

Step	Teammate	Contribution (%)

- Suggested file structure (bonus part is optional):

  assignment_3_final_TEAMMATE1_ TEAMMATE1.zip
  - a3_part_1_ TEAMMATE1_ TEAMMATE2.ipynb
  - a3_part_2_ TEAMMATE1_ TEAMMATE2.ipynb
  - a3_part_3_ TEAMMATE1_ TEAMMATE2.ipynb
  - a3_bonus_visualization_TEAMMATE1_ TEAMMATE2.ipynb
  - a3_bonus_warehouse_TEAMMATE1_ TEAMMATE2.ipynb

Notes:

- Ensure your code is well-organized and includes comments explaining key functions and attributes. Also ensure that all the section headings for your solutions corresponding to this assignment are displayed. After running the Jupyter Notebooks, all results and plots used in your report should be generated and clearly displayed.
- You can submit multiple files, but they all need to be labeled with a clear name.
- Recheck the submitted files, e.g. download and open them, once submitted and verify that they open correctly
- The zip folder should include all relevant files, clearly named as specified.
- Only files uploaded on UBlearns are considered for evaluation.

Academic Integrity

This project can be done in a team of up to two people. Your teammates should be different for A1, A2 and A3.

The standing policy of the Department is that all students involved in any academic integrity violation (e.g. plagiarism in any way, shape, or form) will receive an F grade for the course. The catalog describes plagiarism as “Copying or receiving material from any source and submitting that material as one’s own, without acknowledging and citing the particular debts to the source, or in any other manner representing the work of another as one’s own.”. Refer to the Office of Academic Integrity for more details.

Late Days Policy

You can use up to 7 late days throughout the course that can be applied to any assignment related due dates. You do not have to inform the instructor, as the late submission will be tracked in UBlearns.

If you work in teams, the late days used will be subtracted from both partners. In other words, you have 4 late days and your partner has 3 late days left. If you submit one day after the due date, you will have 3 days and your partner will have 2 late days left.

Important Dates

November 12, Wednesday - Registering your team is Due

November 20, Thursday - Checkpoint is Due

December 4, Thursday - Final Submission is Due